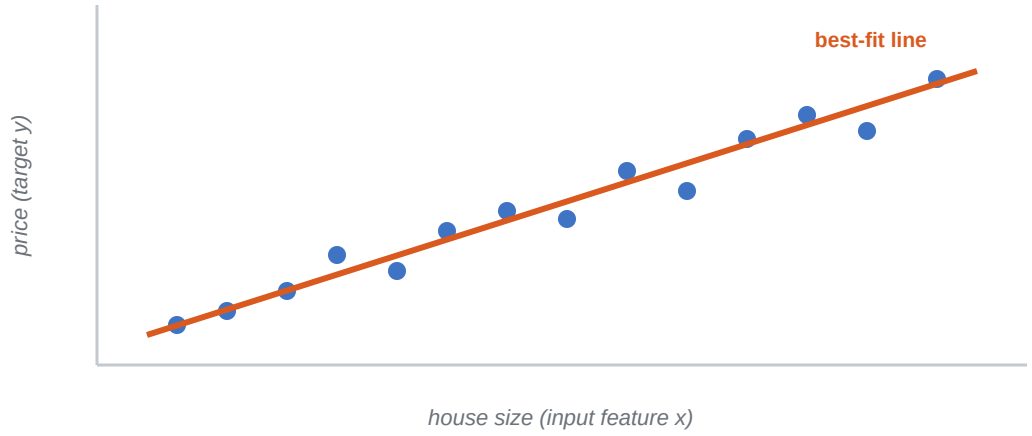
T E C H N I C A L R E F E R E N C E G U I D E

Qwik Guide: Linear Regression

Fitting a Line · Training & Testing · Coefficients · Scoring a Model

Linear regression is the friendliest doorway into machine learning: you give a model some examples, it draws the best straight-line relationship between your inputs and the number you want to predict, and then it predicts that number for data it has never seen. This guide walks through the whole workflow — loading data, splitting it, training a model, reading its coefficients, and measuring how good it is — using a real house-price dataset and defining every term along the way.

WHO THIS IS FOR Anyone who knows a little Python and wants to build and understand their first predictive model. No statistics background required. We use the **scikit-learn** and **Pandas** libraries throughout.



Linear regression finds the straight line that comes closest to all your data points, then uses it to predict new values.

1 What Is Linear Regression?

Machine learning is the practice of getting a computer to learn patterns from examples instead of following rules you write by hand. **Linear regression** is one of the simplest and most widely used machine-learning techniques. Its job is to predict a *continuous* number — a price, a temperature, a weight — from one or more input values.

The idea is to find the straight line (or, with several inputs, the flat "plane") that best fits your data. Once that line is found, you read predictions straight off it. In its simplest one-input form the model is:

$$\text{predicted } y = (\text{slope} \times x) + \text{intercept}$$

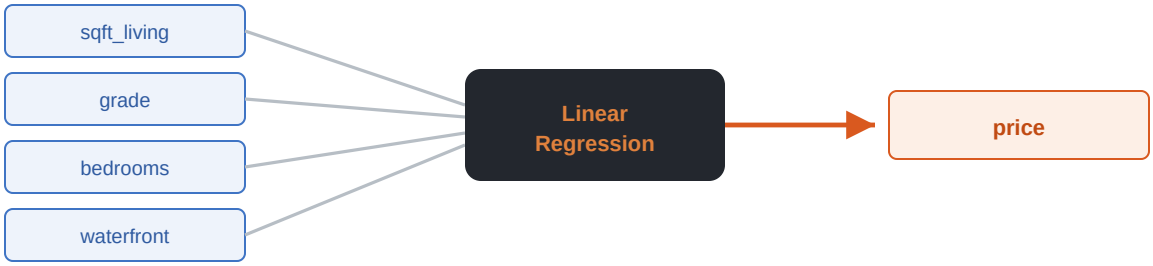
The **slope** tells you how much `y` changes for each one-unit change in `x`, and the **intercept** is where the line crosses the vertical axis (the predicted value when `x` is zero). A **linear regression model** is simply the trained object that has learned the best slope and intercept from your data.

TWO FLAVOURS OF MODEL Models that predict a *number* (like price) are **regression models**. Models that predict a *category* (like "spam / not spam") are **classification models**, or **classifiers**. Both are kinds of machine-learning **estimator** — the general scikit-learn word for any object that learns from data. The preprocessing skills in this guide help both.

Features and labels

Every dataset splits into two roles. The **features** are the input columns the model learns from (house size, grade, zip code). The **label**, or **target variable**, is the single value you are trying to predict (sale price). Choosing which columns to feed the model is called **feature selection**.

features (inputs) → model → label (output)



The model learns a coefficient for each feature, then combines them to predict the label.

The house-price example used throughout

Our worked example predicts the **sale price** of homes in the King County dataset from features such as **grade** (a 1–12 house-condition rating), **zip code**, **waterfront**, **view**, and square footage. Price is the target; everything else is a candidate feature.

FURTHER READING New to the whole idea of fitting a line? Khan Academy's *Linear regression review* is a gentle, visual starting point, and Penn State's *Simple Linear Regression* lesson clearly explains predictors, responses, slope, and intercept. Full links are on the last page.

2 Loading & Exploring the Data

Before modelling anything, load the data and look at it. The **Pandas** library reads a **CSV file** into a **DataFrame** — a table of rows and columns — and gives quick tools for exploration. A single column of a DataFrame is called a **Series**.

- `read_csv()` — loads a CSV file into a DataFrame.
- `head()` — shows the first few rows so you can eyeball the columns.
- `describe()` — summary statistics (count, mean, standard deviation, min, quartiles, max) for every numeric column at once.
- `unique()` — returns the *distinct* values in a column. Great for seeing that `grade` runs 1–12, or how many zip codes exist.
- `isnull()` — flags missing (null) values so you can clean them before training.

```
import pandas as pd
house_data = pd.read_csv("house_prices.csv")
house_data.head()                # peek at the first rows
house_data.describe()            # summary stats for every numeric column
house_data["grade"].unique()   # distinct values → array([7, 6, 8, 11, 9, ... , 12])
house_data[house_data.isnull().any(axis=1)]   # any rows with missing values?
```

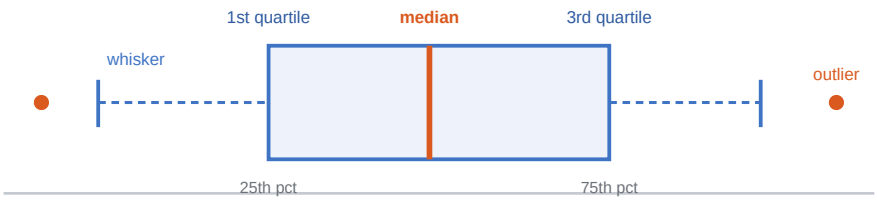
DISTINCT VALUES, THE RIGHT WAY To list the distinct values in a column, the Pandas method is `series.unique()` — for example `house_data["grade"].unique()`. There is no `distinct()` or `unique_values()` method in Pandas; `unique()` is the one to remember.

Shuffling the rows

Raw data often arrives in some order (sorted by date, region, etc.). Before splitting it you usually **shuffle** it so training and test sets are both representative. Pandas' `sample()` can shuffle, and (as you'll see next) `train_test_split` shuffles for you automatically.

Reading a distribution with a box plot

A **box plot** (from Matplotlib's `pyplot`) is a compact picture of a single distribution. It packs a surprising amount of information into one shape:



box = interquartile range (IQR) · whiskers reach 1.5 × IQR · dots beyond = outliers

The box spans the 1st–3rd quartiles, the line inside is the median, whiskers extend 1.5× the IQR, and anything past them is an outlier.

3 Splitting the Data: Train vs. Test

You must never judge a model on the same data it learned from — it could simply memorise the answers. So you divide the data into a **training set** (used to fit the model) and a **test set** (held back to check honest performance on unseen rows). scikit-learn's `train_test_split` does this in one call.

```
from sklearn.model_selection import train_test_split
X = house_data.drop("price", axis=1)     # features: every column except price
y = house_data["price"]                  # label: the value we predict
x_train, x_test, y_train, y_test = train_test_split(
        X, y, test_size=0.2, random_state=0)   # 80% train / 20% test
```

Two arguments matter here. `test_size=0.2` reserves 20% of rows for testing and trains on the other 80%. `train_test_split` also **shuffles** the data first, using a **random number generator**. Setting a **seed** through `random_state` makes that shuffle repeatable, so you (and your readers) get the exact same split every run.

WHY A SEED `random_state=0` doesn't change the quality of your model — it just fixes the shuffle so results are reproducible. Change the number and you get a different (but equally valid) split.

4 Training the Model

With the data split, create a `LinearRegression` estimator and call `.fit()`, passing the training features and training labels. Fitting *is* the learning step: the model searches for the slope and intercept (in general, one **coefficient** per feature) that make the line fit the training points as closely as possible.

```
from sklearn.linear_model import LinearRegression
linear_model = LinearRegression()          # default settings
linear_model.fit(x_train, y_train)         # learn from the training data
```

fit_intercept By default the model also learns an **intercept** (`fit_intercept=True`), which is usually what you want. Later, when we one-hot encode a column, we'll need to avoid a **collinearity** problem — normally by dropping one dummy column — more on that on the next page.

Reading the coefficients

After fitting, the learned weights live in the `coef_` attribute — one number per feature. A large *positive* coefficient pushes the predicted price up; a large *negative* one pulls it down. Pairing each coefficient with its column name turns raw numbers into insight.

```
coefficients = pd.Series(linear_model.coef_, x_train.columns).sort_values()
print(coefficients)
# waterfront & grade → strong positive effect on price
# bedrooms   → negative here (pricier homes are smaller city condos)
```

In our house data, **waterfront** and **grade** carry big positive coefficients (they raise the price), while a higher **bedroom** count actually correlates with a *lower* price — a hint that the priciest homes are smaller, well-located condos rather than sprawling houses.

5 Preprocessing the Features

A first model on raw data works, but usually under-performs. Two preprocessing moves reliably help linear regression: **scaling** the continuous features so no single big-range column dominates, and **encoding** the categorical ones into numbers.

Scaling continuous features

Features like square footage (thousands) and grade (1–12) live on wildly different scales. **Min-max scaling** (`MinMaxScaler`) squeezes each column into a fixed range — by default 0 to 1 — so they all carry comparable weight.

```
from sklearn.preprocessing import MinMaxScaler
scaler = MinMaxScaler()                       # default range 0–1
scaled_x = scaler.fit_transform(x_continuous)  # returns a NumPy array, all values in [0, 1]
```

The `fit_transform` method does two jobs at once: it *fits* (learns each column's min and max) and *transforms* (rescales the values). You can inspect a scaler's settings with `get_params()`. For the full family of scalers — `StandardScaler`, `MaxAbsScaler`, the row-wise `Normalizer` — see the companion *Data Preprocessing* guide.

> **FIT ON TRAINING DATA** A scaler learns its min/max (or mean/std) from the data you fit it on. Fit on the **training set only**, then apply that same transform to the test set — otherwise information from the test data leaks into training.

5.1 Encoding Categorical Features

A column like zip code holds **categorical values** — labels, not quantities. Feeding raw category numbers would fool the model into thinking one zip code is "greater" than another. **One-hot encoding** gives each category its own 0/1 column. Pandas' `get_dummies()` does it in a single call — the form you use depends on whether you pass a whole **DataFrame** or a single **Series**.

```
import pandas as pd
# one-hot encode the "city" column of a DataFrame called homes:
pd.get_dummies(homes, columns=["city"])   # DataFrame → encodes just city, keeps other columns
pd.get_dummies(homes["city"])             # Series → one column per category
```

MIND THE `columns=` ARGUMENT `columns=` is a **DataFrame-only** parameter — it names which columns to encode. If you pass it alongside a single Series, as in `pd.get_dummies(homes["city"], columns=["city"])`, Pandas *silently ignores it*. You still get the right output, but the argument is meaningless there. Write `pd.get_dummies(homes["city"])` for a Series and reserve `columns=` for a DataFrame.

One-hot encoding: one column per category

A single "MattressSize" column with four values becomes four 0/1 columns — one 1 per row marks the class. No false ordering is implied.

WHEN LABEL ENCODING ISN'T ENOUGH With only two categories you could **label-encode** to a single 0/1 column. But with **more than two** distinct values (Full, King, Queen, Twin), label encoding invents a fake order — use **one-hot encoding** via `get_dummies()` instead.

ONE-HOT & THE DUMMY-VARIABLE TRAP A full set of one-hot columns always sums to 1, so together with an intercept they are perfectly redundant — the **dummy-variable trap**, a form of **collinearity**. The usual fix is to **drop one dummy column** per category (`pd.get_dummies(..., drop_first=True)`), which keeps the intercept meaningful. Dropping the intercept instead with `LinearRegression(fit_intercept=False)` is an alternative you'll see in tutorials, but it is not a universal rule — and it stops working once you one-hot encode *two or more* categorical columns, since each set still sums to 1. Finally, `pd.concat([...], axis=1)` stitches your scaled continuous frame and encoded categorical frames back into one before re-training.

6 Evaluating the Model

A trained model is only useful if it predicts well on *unseen* data. First, generate predictions on the test set with `.predict()`; then score them.

```
y_predict = linear_model.predict(x_test)        # predicted prices for the held-out rows
r_square  = linear_model.score(x_test, y_test)  # R² on the test data
```

The score() method returns R²

For a `LinearRegression` model, the `.score()` method returns the **R² (R-square) coefficient** of the prediction — a number that says how much of the variation in price the model explains. It ranges up to 1.0, where higher is better. An R² of 0.65 means the model explains about 65% of the variance — decent, with room to improve through the scaling and encoding on the previous page.

R² = 1 − (model's squared error ÷ error of always guessing the mean)

R² HAS A CATCH Adding more features can only push R² up, even useless ones. **Adjusted R²** corrects for this by penalising extra features — if a new feature doesn't truly help, adjusted R² can actually fall. Prefer it when comparing models with different feature counts.

6.1 Error Metrics & Visual Checks

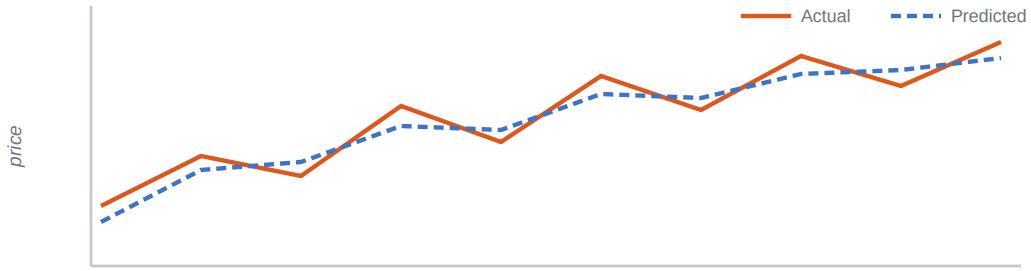
R^2 measures explained variance; error metrics measure how far off predictions are, in price units.

Metric	What it measures	Notes
MSE Mean Squared Error	Average of the squared gaps between predicted and actual price.	The usual **loss function** for regression. Lower is better; big errors are punished heavily.
RMSE Root Mean Sq. Error	The square root of MSE.	Back on the same scale as price, so easier to interpret. Use Python's `math.sqrt()`.
MAE Mean Absolute Error	Average of the *absolute* gaps.	Already in price units; less sensitive to a few huge misses than MSE.
R^2 R-square score	Fraction of variance the model explains (0→1).	Returned by `.score()`. Higher is better; see adjusted R^2 above.

```
from sklearn.metrics import mean_squared_error
import math
mse  = mean_squared_error(y_test, y_predict)
rmse = math.sqrt(mse)                  # RMSE — same units as price
```

Eyeballing predictions

Plotting predicted vs. actual prices (say, the first 150 test homes) gives a quick visual gut-check: if the two lines track each other, your model is capturing the real pattern — matching a healthy R^2 and a low error.



When the predicted line (blue) closely shadows the actual line (orange), the model is doing its job.

★ Quick Reference

The tools in this guide come from **two different libraries**. The **Library** column below tells you which is which — **Pandas** handles loading, inspecting, and encoding your data, while **scikit-learn** handles splitting, scaling, training, and scoring the model.

Tool / method	Library	What it does	Result
`read_csv()`	**Pandas**	Load a CSV into a DataFrame	A table of rows and columns
`head()`	**Pandas**	Show the first few rows	A preview of the DataFrame
`describe()`	**Pandas**	Summarise every numeric column	count, mean, std, min, quartiles, max
`unique()`	**Pandas**	List distinct values in a column	Array of the unique values
`isnull()`	**Pandas**	Flag missing values	True/False per cell
`get_dummies()`	**Pandas**	One-hot encode a category column (`columns=` is DataFrame-only)	One 0/1 column per category; `drop_first=True` drops one
`concat()`	**Pandas**	Join frames back together (`axis=1`)	One unified DataFrame
`train_test_split()`	**scikit-learn**	Shuffle & split into train/test	x_train, x_test, y_train, y_test
`MinMaxScaler`	**scikit-learn**	Scale continuous columns to a range	Default 0–1
`StandardScaler`	**scikit-learn**	Standardize a column	Mean 0, std 1 (Z-scores)
`LinearRegression()`	**scikit-learn**	Create the regression estimator	An unfitted model
`.fit(x, y)`	**scikit-learn**	Learn slope/intercept from training data	A fitted model
`.coef_`	**scikit-learn**	Learned weight for each feature	Array of coefficients
`.predict(x)`	**scikit-learn**	Predict labels for new inputs	Array of predicted values
`.score(x, y)`	**scikit-learn**	Evaluate the fitted model	R^2 (coefficient of determination)
`mean_squared_error()`	**scikit-learn**	Measure average squared prediction error	MSE (take `sqrt` for RMSE)

RULE OF THUMB If it works on the *data* — reading, previewing, encoding, joining — reach for **Pandas** (`import pandas as pd`). If it works on the *model* — scaling for training, fitting, predicting, scoring — reach for **scikit-learn** (`from sklearn... import ...`).

★ Learn More

Six places to take linear regression further.

Khan Academy — Linear regression review
A gentle, visual introduction.
`https://www.khanacademy.org/math/statistics-probability/describing-relationships-quantitative-data/introduction-to-trend-lines/a/linear-regression-review`

Penn State — Simple Linear Regression
Clearly explains predictors, responses, slope, intercept, and predictions.
`https://online.stat.psu.edu/stat200/lesson/12/12.3`

GeoGebra — Interactive Linear Regression
Experiment with data points and try fitting a line yourself.
`https://www.geogebra.org/m/xC6zq7Zv`

Seeing Theory — Regression Analysis
Interactive explanation of least-squares regression.
`https://seeing-theory.brown.edu/regression-analysis/`

Google Machine Learning Crash Course — Linear Regression
Introduces the machine-learning perspective, including loss and gradient descent.
`https://developers.google.com/machine-learning/crash-course/linear-regression`

Penn State — Complete Simple Linear Regression Lesson
A more thorough next step covering residuals, R^2, assumptions, and examples.
`https://online.stat.psu.edu/stat501/Lesson01`